



Birzeit University

Faculty of Engineering and Technology

Department of Electrical and Computer Engineering

ENCS3320 – Computer Networks (Term 1243)

Project #1 (Socket Programming) – Due Thursday, August 07, 2025

A) Objectives

This project aims to deepen your understanding of socket programming and computer networking. Specifically, the objectives are as follows:

- Understand how to create Transmission Control Protocol (TCP) and User Datagram Protocol (UDP) sockets.
- Learn the basics of HyperText Markup Language (HTML) and web server configuration.
- Develop teamwork skills by collaborating on project tasks.

B) Requirements and Deliverables

This project includes *two tasks*. As a team of *three students*, you are responsible for completing **all** tasks and submitting your source codes. Please consider the following requirements:

- **Implementation Guidelines:** You may choose *any programming language* (e.g., Python, Java, C, etc.). Note that libraries should not be used for implementing socket functionalities; *only the standard socket library is allowed*. The implementation should reflect *your own work* and be adaptable for future expansion with minimal effort (i.e., your implementation should be as generic as possible).
- **Team Coordination:** Each team member should actively participate across all project stages, including research, design, implementation, testing, and report preparation. To support collaboration, we recommend using GitHub for version control and collaborative coding, and Overleaf for seamless, real-time document editing. Consider dividing responsibilities clearly, sharing regular feedback, and tracking progress closely to ensure a unified and successful project outcome.

Please submit the following **files** as *a single compressed folder* (.zip) named **Project1_TeamName.zip** through ITC (<https://itc.birzeit.edu/course/view.php?id=34596>):

- 1) **Task 1 Folder:** A folder named **Task1** containing **well-documented source code and files** for Task 1. This folder should include: (i) The main server code (e.g., `server.py`) and (ii) Subfolders named `html`, `css`, and `imgs` containing the necessary HTML, CSS, and image files for the server, respectively. Include as many subfolders as needed to make your implementation well-organized.
- 2) **Task 2 Folder:** A folder named **Task2** with **well-documented source code** for running the server (e.g., `server.py`) and client (e.g., `client.py`) for Task 2.

Each team should submit **one** final version. The submission **deadline** is Thursday, August 07, 2025.

Helpful resources to support your success on this project are available at the following link: https://drive.google.com/drive/folders/1bB6hX3MUIGcjFe6BfG8vUR3_eg3VPLtc?usp=sharing.

C) Project Tasks

1) Task 1 – Web Server

Using socket programming, implement a simple but complete web server that listens on port **5012**. The server should support the following HTML web pages:

a. Main English Webpage (*main_en.html*):

This page should include:

- The text "ENC3320-T1243" displayed on the web browser tab.
- The title "Welcome to Computer Networks Webserver" at the top of the page.
- Team members' photos (in **.png** format) displayed alongside their names and IDs, arranged in a table-like structure. Organize each member's information within separate boxes on the page.
- A brief description of each team member, covering details like projects completed in various courses, skills, hobbies, etc.
- A section that provides an informative overview highlighting the suffering of our people in Gaza due to the ongoing war. The content should convey the fear of losing lives, the sadness caused by the loss of loved ones, the difficulty in accessing food and water, and the hardships resulting from repeated displacement. This section must include elements such as well-structured paragraphs, relevant images (in **.jpg** format), and both ordered and unordered lists. Use appropriate headings, font styles, colors, and text formatting (e.g., bold and italic) to enhance readability and emphasize key points.
- A form where users can request details about a specific event during the war on Gaza by entering text into an input box. The server should respond by returning a **local HTML file** (named *event_details.html*) that includes documentation about the event, along with relevant image(s) and/or video(s) associated with the keyword. The server must have at least one such local HTML file available.

If the requested topic is unavailable, the server should respond with a "**307 Temporary Redirect**" status code and redirect the client (i.e., the web browser) based on the type of content:

- **Text:** Redirect to an Al Jazeera Media Network search URL that filters results based on the user's input (e.g., <https://www.aljazeera.com/search/War%20crimes%20Gaza> for the input "*War crimes Gaza*").
- **Images:** Redirect to a Google search URL that filters results to images based on the user's input (e.g., <https://www.google.com/search?q=gaza+humanitarian+crisis&udm=2> for the input "*gaza humanitarian crisis*").
- **Videos:** Redirect to a YouTube search URL that filters results to videos based on the user's input (e.g., https://www.youtube.com/results?search_query=Al-Ahli+Arab+Hospital+bombing for the input "*Al-Ahli Arab Hospital bombing*").
- Links to external resources, including:
 - The textbook website (https://gaia.cs.umass.edu/kurose_ross/index.php)
 - Ritaj website (<https://ritaj.birzeit.edu/>)
- A separate CSS file to style the page attractively.

b. Arabic Versions:

Provide Arabic versions of the *main_en.html* and *event_details.html* files, named *main_ar.html* and *event_details.html_ar.html*, respectively.

Server Functionality:

When the server receives an HTTP request, it should:

- Print the HTTP request details to the terminal.

- Generate and send an HTTP response that includes the requested content and specifies the correct Content-Type based on the file type (e.g., `text/html` for HTML, `text/css` for CSS, `image/png` for PNG, and `video/mp4` for MP4).

Default and Error Responses:

- For requests targeting `/`, `/en`, `/index.html`, or `/main_en.html` (e.g., <http://localhost:5012/>, <http://localhost:5012/en>, <http://localhost:5012/index.html>, or http://localhost:5012/main_en.html), the server should respond with `main_en.html`.
- For requests targeting `/ar` or `/main_ar.html` (e.g., <http://localhost:5012/ar> or http://localhost:5012/main_ar.html), the server should respond with `main_ar.html`.
- If the client requests an invalid URL (i.e., a file not found on the server), the server should return a simple HTML error page with:
 - The status line "HTTP/1.1 404 Not Found"
 - The text "Error 404" in the browser tab
 - The message "The file is not found" displayed in red text in the body
 - The client's IP address and port number.

2) Task 2 – UDP Client-Server Multiplayer Number Game

In this task, you are required to implement a **multiplayer number submission game** using **UDP socket programming**. The game is designed to test the clients' attentiveness and strategic thinking in a dynamic, round-based environment. The server oversees the entire game flow, ensuring fairness, managing the state of connected clients, and enforcing the rules consistently throughout the game.

The game begins only when **at least two active clients** are connected. Clients may **join or leave at any time**, even after the game has started. The gameplay consists of multiple rounds. In each round, every client must submit a **unique number between 1 and 100** that has **not been submitted by any other client** in the current or previous rounds. Clients **do not see** the numbers submitted by others. If a client submits a number that has already been used, they **immediately lose**, and the server closes their connection. The game continues until one of two conditions is met: **only one client remains**, or **all valid numbers have been used** while multiple clients are still connected. In both cases, all remaining clients are declared **winners**.

a. Server Responsibilities

- **Connection Management:** The server listens on port **5012** for incoming UDP connections. Each client is uniquely identified by their IP address and port number. The server maintains an active list of connected clients.
- **Game Initialization:** The game begins only when at least two clients are connected. The server then broadcasts a message announcing the start of the game.
- **Round Management:** At the beginning of each round:
 - The server sends a request to all active clients to submit a number between **1** and **100** that has not been used before.
 - The server enforces a time limit (e.g., **60** seconds) for submissions.
 - It verifies the uniqueness of each number submitted across all previous rounds.
- **Client Elimination:** If a client submits a previously used number, the server notifies them of their elimination and closes their connection.
- **End-of-Game Detection:** Before starting a new round, the server checks:
 - If only one client remains → declare them a winner.
 - If no more unused numbers exist and clients still remain → declare all remaining clients winners.

The server broadcasts a termination message to all remaining clients and gracefully shuts down the game.

- **Broadcast Messages:** After each round, the server may broadcast an update on the number of remaining players and remaining available numbers.

b. Server Terminal Output

- Confirmation that the server has started and is listening on port 5012.
- Notification when each client connects or disconnects, along with their IP and port.
- Start of each round and broadcast to clients.
- Display of submitted numbers along with client identifiers and validation results.
- Notification when a client is eliminated.
- Announcement of game termination and list of winners.
- Server shutdown message.

c. Client Responsibilities

- **Connecting to the Server:** The client connects to the server using the server's IP address and port number. Upon joining, they submit a username for identification.
- **Game Participation:**
 - Clients may join or leave at any time.
 - At the beginning of each round, they will receive a prompt to submit a number between **1** and **100**.
 - Clients must ensure they submit a number they have not used before and that no other client has submitted previously (though they are not told the history).
 - Clients who submit a used number are eliminated by the server.
- **Receiving Notifications:** Clients receive messages from the server about round start, submission status, eliminations, and game termination.

d. Client Terminal Output

- Confirmation of successful connection to the server.
- Prompts to enter a number at the start of each round.
- Confirmation of number submission.
- Notification if the submitted number is invalid or has already been used.
- Notification of elimination or victory.
- Notification of server shutdown.

D) Grading Criteria

This project is worth 15% of the total grade: 12% will be allocated to the source code, while the remaining 3% will be allocated to the project discussion (short-answer questions related to this project). Performance on this project will also be partially competitive, with additional points awarded for the following distinguishing factors:

- Quality and completeness of functioning features.
- The cohesiveness of the integrated components in achieving the project objectives.
- Demonstrated understanding of implementation details.
- User-friendliness and ease of interfacing with the components.
- Deductions may apply if extensions or names deviate from those specified in [Section B \(Requirements and Deliverables\)](#) and [Section C \(Project Tasks\)](#).

GOOD LUCK